20220702045

Ece Düzgeç

Link for Youtube Video: https://youtu.be/xjQwCb-9fQQ

According to the C++ code inside the link. First we create the matrix and function to create a random matrix with a specific seed and generate random numbers between 0 and 10 and create an n×n matrix filled with filling with a random numbers.

Then we create function for Brute Force Matrix Multiplication. For that we get the size of the matrix and create a result matrix filled with zeros. Then for each row and column we multiply and add them.

For Brute Force Matrix Multiplication to get cell C[i][j], we multiply row i of matrix A with column j of matrix B by doing this for every cell in the result matrix

Later we create Naive Divide and Conquer Matrix Multiplication function by using the base the base case and splitting the matrix into 4 parts. We use half the size and get 8 parts .We copy data of these parts and by using helper functions and recursion we add and substract two matrices. And combine these 4 parts back into one.

For Naive Divide and Conquer, it is like a recursive approach, as we split both matrices into 4 parts and multiply the quarters using the same function to add the results together to get them back into a full.

For the Strassen Multiply function we do the same steps as we did for Divide & Conquer but instead of 8 multiplications, Strassen uses 7 and combines these 7 parts into 4 parts and the combine these 4 into one.

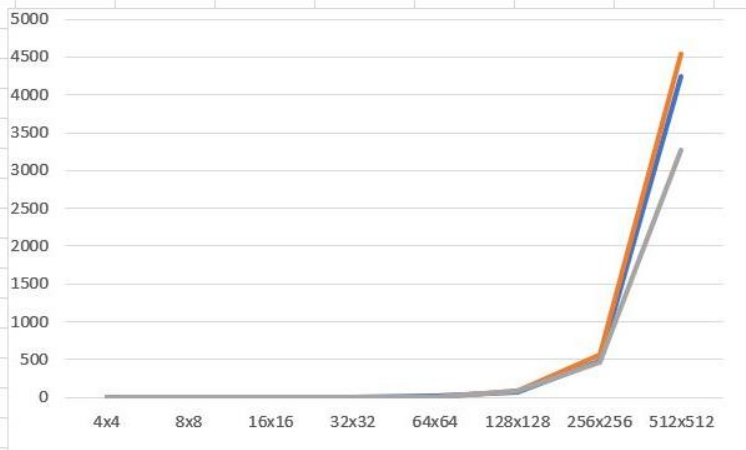
Afterwards, for the Performance Measurement we calculate the duration and convert it to milliseconds. As the firrst run is ignored for accuracy we run the code multiple times and return average time.By using output to 4 decimal places and average of 5 runs for each test, we test each algorithm and print it.

Brute Force Matrix Multiplication has O(n^3) time complexity as it is a simple triple nested loop and best for small matrices such as n<128. In 2 Seconds Maximum Matrix Size achievable is ~512x512 to 768x768.

Naive Divide and Conquer has O(n^3) time complexity and uses recursive approach with 8 subproblems. It is generally slower than brute force due to recursion overhead. In 2 Seconds Maximum Matrix Size achievable is ~512x512 to 768x768.

Strassen's Algorithm has O(n^2.807) time complexity. It approaches with 7 subproblems with recursively. Becomes efficient for larger matrices as in 2 Seconds Maximum Matrix Size achievable is ~1024x1024 to 1536x1536.

Size	Brute Force	Naive D&C	Strassen's
4x4	0.0063	0.0155	0.0095
8x8	0.064	0.0464	0.0383
16x16	0.1372	0.1395	0.1914
32x32	1.9819	1.8924	0.9931
64x64	12.2637	10.6469	10.5514
128x128	66.9904	79.1316	80.3097
256x256	487.8265	564.5198	463.0785
512x512	4237.144	4543.9299	3275.7816



According to the results from a test run. We can refer that :

For small matrices (4x4 to 16x16): Though all algorithms have similar performance (< 1ms) Brute Force is the simplest and fastest as it avoids recursive methods due to overhead. The overhead of recursion makes D&C and Strassen slower

For medium matrices (32x32 to 128x128): Even though Brute Force maintains its advantage. Naive D&C is consistently slower than Brute Force due to recursion overhead. Strassen's begins to show advantages

For Large matrices (256x256 to 512x512): Strassen's Algorithm clearly better as it can handle significantly larger problems.

So, we can see that for very large matrices (n > 1024): Only Strassen's Algorithm remains practical but we may need to consider memory optimized implementation and may need specialized libraries.

So overall, for small matrices using Brute Force, for medium matrices transitioning to Strassen's and for large matrices Strassen's Algorithm is essential. And for very large matrices combination of Strassen's with advanced optimization techniques is more efficient. Since, Strassen's algorithm provides ~2-3x improvement for large matrices like n >= 256 and Naive D&C has minimal practical value due to overhead.